COS101 Practical — Week 2: Variables, Data Types, User Input, and Arithmetic

Programme: B.Sc. Software Engineering
Level: 100
Duration: 3 hours
Prerequisite: Ability to create, save, and run a Python script, as covered in Week 1.

This week, you will extend your earlier programs so that you can store information, accept values from users, perform calculations, and display clear results.

1. Learning objectives

By the end of this practical, you should be able to:

1. Create variables using meaningful names.
2. Explain and use integers, floating-point numbers, strings, and Boolean values.
3. Inspect a value's data type using `type()`.
4. Accept keyboard input using `input()`.
5. Convert input into numbers using `int()` and `float()`.
6. Apply arithmetic operators and operator precedence correctly.
7. Display labelled results using f-strings.
8. Build simple arithmetic, temperature-conversion, and simple-interest programs.
9. Recognise common input and calculation errors.
10. Check program results against manual calculations.

2. Materials and preparation

What you need:

- A computer with Python 3.
- IDLE or the editor used in Week 1.
- Their `COS101_Practicals` folder.
- A notebook for calculations and observations.

Create a new folder named `Week02` inside `COS101_Practicals`.

Save this week's programs there, using filenames such as:

- `variables_demo.py`
- `student_profile.py`
- `rectangle_calculator.py`
- `basic_calculator.py`
- `temperature_converter.py`
- `simple_interest.py`

Working habit: Predict the result, save the code, run it, and compare the actual output with your prediction.

3. From fixed values to reusable programs

In Week 1, we calculated the area of a rectangle using fixed numbers:

```
print("Area:", 12 * 7, "square metres")
```

This works, but calculating a different rectangle requires editing the program.

We can improve it in two stages:

1. Store the dimensions in variables.
2. Ask the user to enter the dimensions.

This is the foundation of programs that respond to different situations.

4. Understanding variables

A **variable** is a name that refers to a value.

For example:

```
length = 12
width = 7
```

Here:

- `length` refers to the integer `12`.

- `width` refers to the integer `7`.

The `=` symbol is the **assignment operator**. It assigns the value on the right to the name on the left.

Read:

```
length = 12
```

as:

> Assign the value 12 to the variable named length.

It does not mean “test whether length equals 12.” Comparisons will be introduced in Week 3.

4.1 Using variables in calculations

```
length = 12
width = 7
area = length * width

print("Area:", area, "square metres")
```

Expected output:

```
Area: 84 square metres
```

Python evaluates `length * width` and assigns the result to `area`.

4.2 Updating a variable

A variable can be assigned a new value:

```
score = 40
print("Original score:", score)

score = 65
print("Updated score:", score)
```

Expected output:

```
Original score: 40
Updated score: 65
```

Assignment does not automatically update earlier calculations:

```
length = 12
width = 7
area = length * width

length = 20

print("Area:", area)
```

The output remains:

```
Area: 84
```

To calculate the new area, execute the calculation again:

```
area = length * width
print("New area:", area)
```

Expected output:

```
New area: 140
```

4.3 Variable naming rules

For the names used in this course:

- Begin with a letter or underscore.
- Use letters, digits, and underscores.
- Do not include spaces or hyphens.
- Do not use Python keywords such as `if`, `for`, or `class`.
- Remember that names are case-sensitive.

Name	Valid?	Explanation
`student_name`	Yes	Clear and descriptive
`score1`	Yes	A digit can appear after the first character

`total_cost`	Yes	Uses an underscore between words
`1score`	No	Begins with a digit
`student name`	No	Contains a space
`total-cost`	No	The hyphen is interpreted as subtraction
`class`	No	A Python keyword

Use lowercase names with underscores between words:

```
unit_price = 250
number_of_pens = 4
total_cost = unit_price * number_of_pens
```

Avoid replacing built-in names such as `print`, `input`, `int`, or `float` with your own variables.

Practical Activity 1: Variables

Save as `variables_demo.py`:

Calculate the cost of buying notebooks.

```
item_name = "Notebook"
quantity = 3
unit_price = 500
total_cost = quantity * unit_price

print("Item:", item_name)
print("Quantity:", quantity)
print("Unit price:", unit_price, "naira")
print("Total cost:", total_cost, "naira")
```

Expected output:

```
Item: Notebook
Quantity: 3
Unit price: 500 naira
```

```
Total cost: 1500 naira
```

Tasks:

1. Change the quantity to `5`.
2. Predict the new total before running.
3. Run the program.
4. Identify which variables store text and which store numbers.

5. Basic data types

A **data type** describes the kind of value being used.

This week introduces four basic types:

Type	Python name	Description	Examples
Integer	`int`	Whole numbers	`100`, `0`, `-5`
Floating-point number	`float`	Numbers represented using floating-point arithmetic	`3.5`, `250.0`, `-2.75`
String	`str`	Text enclosed in quotation marks	`"Python"`, `"COS101"`
Boolean	`bool`	A truth value	`True`, `False`

5.1 Integers

Use integers for whole-number quantities:

```
number_of_students = 45
number_of_chairs = 48
```

Write the number without quotation marks.

5.2 Floating-point numbers

Use floats when values may contain fractions:

```
temperature = 36.5
length = 12.75
```

A decimal point makes a numeric literal a float:

```
price = 500.0
```

Floating-point arithmetic can produce small rounding differences:

```
print(0.1 + 0.2)
```

Output:

```
0.30000000000000004
```

This results from how many decimal fractions are represented internally. Later in this note, we will format results for readable display.

5.3 Strings

Strings represent text:

```
student_name = "Ada Okeke"
course_code = "COS101"
```

A number inside quotation marks is also a string:

```
matriculation_number = "00125"
```

This is appropriate for identifiers because we do not normally perform arithmetic on them, and leading zeros may matter.

Compare:

```
print(10 + 5)
print("10" + "5")
```

Output:

```
15
105
```

For numbers, `+` performs addition. For strings, it joins the text; this is called **concatenation**.

5.4 Boolean values

Booleans represent true or false:

```
is_registered = True
has_paid_fees = False
```

`True` and `False` begin with capital letters and have no quotation marks.

```
True
```

is a Boolean, whereas:

```
"True"
```

is a string.

We will use Boolean expressions extensively when studying decisions in Week 3.

5.5 Inspecting types with `type()`

```
student_name = "Ada Okeke"
age = 18
height = 1.65
is_registered = True

print(type(student_name))
print(type(age))
print(type(height))
print(type(is_registered))
```

Expected output:

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

Practical Activity 2: Identify the types

Predict the type of each value, then check with `type()`:

```
course_code = "COS101"
level = 100
average_score = 72.5
is_present = True
phone_number = "08012345678"
```

Discussion: Why is a phone number better stored as text than as an integer?

6. Accepting user input

The `input()` function displays a prompt and waits for the user to type a response and press Enter.

```
student_name = input("Enter your full name: ")
print("Welcome,", student_name)
```

Example interaction:

```
Enter your full name: Ada Okeke
Welcome, Ada Okeke
```

The text typed by the user becomes the value assigned to `student_name`.

Key rule: `input()` returns a string, even when the user types digits.

Practical Activity 3: Interactive student profile

Save as `student_profile.py`:

Collect and display a student's academic information.

```
student_name = input("Enter your full name: ")
matriculation_number = input("Enter your matriculation
number: ")
programme = input("Enter your programme: ")

print()
print("STUDENT PROFILE")
```

```
print("===============")
print("Name:", student_name)
print("Matriculation number:", matriculation_number)
print("Programme:", programme)
print("Level: 100")
```

Run the program twice using different information.

Observation: The program now produces different results without requiring changes to its source code.

7. Converting input into numbers

Consider this program:

```
first_number = input("Enter the first number: ")
second_number = input("Enter the second number: ")

total = first_number + second_number

print("Total:", total)
```

If the user enters `10` and `5`, the result is:

```
Total: 105
```

Both inputs are strings, so Python joins them.

To perform arithmetic, convert the input to an appropriate numeric type.

7.1 Converting to integers with `int()`

```
first_number = int(input("Enter the first whole number: "))
second_number = int(input("Enter the second whole number: "))

total = first_number + second_number

print("Total:", total)
```

Entering `10` and `5` now gives:

Total: 15

The first line performs three steps:

1. `input()` collects text.
2. `int()` converts that text into an integer.
3. The integer is assigned to `first_number`.

7.2 Converting to floats with `float()`

Use `float()` when fractional values are allowed:

```
length = float(input("Enter the length in metres: "))
width = float(input("Enter the width in metres: "))

area = length * width

print("Area:", area, "square metres")
```

Entering `4.5` and `3` gives:

```
Area: 13.5 square metres
```

7.3 Choosing a conversion

Information	Appropriate approach
Student name	Keep the string returned by `input()`
Matriculation number	Keep it as a string
Number of chairs	Convert with `int()`
Temperature	Convert with `float()`
Length or width	Convert with `float()`
Price in these introductory exercises	Convert with `float()`

7.4 Invalid conversions

These conversions work:

```
int("12")
float("12")
float("12.5")
```

These raise a `ValueError`:

```
int("twelve")
int("12.5")
float("five")
```

For this week, provide valid numerical input when prompted. We will introduce validation in Week 3 and exception handling in Week 9.

Also note:

```
int(12.9)
```

produces 12. Converting a float to an integer truncates towards zero; it does not round to the nearest integer.

8. Arithmetic operators

Python supports the following arithmetic operators:

Operator	Meaning	Example	Result
`+`	Addition	`9 + 4`	`13`
`-`	Subtraction	`9 - 4`	`5`
`*`	Multiplication	`9 * 4`	`36`
`/`	Ordinary division	`9 / 4`	`2.25`

`//`	Floor division	`9 // 4`	`2`
`%`	Remainder	`9 % 4`	`1`
`**`	Exponentiation	`3 ** 2`	`9`

8.1 Ordinary division

```
print(10 / 2)
```

Output:

```
5.0
```

Ordinary division returns a float, even when the division is exact.

8.2 Floor division and remainder

Suppose 17 students are placed in groups of 5:

```
students = 17
group_size = 5

complete_groups = students // group_size
remaining_students = students % group_size

print("Complete groups:", complete_groups)
print("Students remaining:", remaining_students)
```

Output:

```
Complete groups: 3
Students remaining: 2
```

For positive integers, floor division tells us how many complete groups fit, and remainder tells us how many are left.

More generally, floor division rounds the quotient down:

```
print(-7 // 2)
```

Output:

```
-4
```

8.3 Exponentiation

```
print(5 ** 2)
print(2 ** 3)
```

Output:

```
25
8
```

Use `**` for powers. The symbol `^` does not mean exponentiation in Python.

Practical Activity 4: Convert minutes

Save as `minutes_converter.py`:

Convert a whole number of minutes into hours and remaining minutes.

```
total_minutes = int(input("Enter the total number of minutes: "))

hours = total_minutes // 60
remaining_minutes = total_minutes % 60

print("Hours:", hours)
print("Remaining minutes:", remaining_minutes)
```

Test with:

Input	Expected hours	Expected remaining minutes
45	0	45
60	1	0
135	2	15

Use non-negative whole numbers for this exercise.

9. Operator precedence

Python follows an order when evaluating arithmetic expressions.

For the expressions used in this lesson:

1. Parentheses.
2. Powers.
3. Multiplication, division, floor division, and remainder.
4. Addition and subtraction.

Operators in the multiplication group are evaluated left to right. Addition and subtraction are also evaluated left to right.

Compare:

```
print(2 + 3 * 4)
print((2 + 3) * 4)
```

Output:

```
14
20
```

In the first expression, multiplication happens before addition. In the second, the parentheses cause addition to happen first.

9.1 A common average-calculation mistake

Incorrect:

```
average = 60 + 70 + 80 / 3
```

Only `80` is divided by `3`.

Correct:

```
average = (60 + 70 + 80) / 3
```

Practical Activity 5: Predict before running

Write down the result of each expression, then check in Python:

```
print(10 + 2 * 3)
print((10 + 2) * 3)
print(20 / 5 * 2)
print(2 ** 3 + 1)
print(17 // 5)
print(17 % 5)
```

Expected results, in order:

```
16
36
8.0
9
3
2
```

10. Displaying results with f-strings

An **f-string** allows you to insert values into text.

Write `f` immediately before the opening quotation mark, and place variables or expressions inside braces `{}`.

```
student_name = "Ada Okeke"
level = 100

print(f"{student_name} is a {level}-level student.")
```

Output:

```
Ada Okeke is a 100-level student.
```

10.1 Formatting decimal places

```
amount = 1250.5

print(f"Amount: {amount:.2f} naira")
```

Output:

```
Amount: 1250.50 naira
```

The format `:.2f` displays a number in fixed-point notation with two digits after the decimal point.

To include thousands separators:

```
amount = 12500.5

print(f"Amount: NGN {amount:,.2f}")
```

Output:

```
Amount: NGN 12,500.50
```

This formatting changes how the number is displayed. It does not change the value stored in `amount`.

If you forget the `f`:

```
print("Amount: {amount:.2f}")
```

Python displays the braces and their contents literally.

Practical Activity 6: Improve the rectangle program

Save as `rectangle_calculator.py`:

Calculate area and perimeter from dimensions entered by the user.

```
length = float(input("Enter the length in metres: "))
width = float(input("Enter the width in metres: "))
```

```
area = length * width
perimeter = 2 * (length + width)

print()
print("RECTANGLE CALCULATION")
print("====================")
print(f"Length: {length:.2f} metres")
print(f"Width: {width:.2f} metres")
print(f"Area: {area:.2f} square metres")
print(f"Perimeter: {perimeter:.2f} metres")
```

For length 12 and width 7, the calculated results should be:

```
Area: 84.00 square metres
Perimeter: 38.00 metres
```

Check the units: Area uses square metres; perimeter uses metres.

11. Guided project 1: Basic arithmetic calculator

Filename: `basic_calculator.py`

Problem

Create a program that accepts two numbers and displays their sum, difference, product, and quotient.

For this version, the second number must be non-zero.

Input–process–output analysis

Input	Process	Output
Two numbers	Add, subtract, multiply, and divide	Four labelled results

Pseudocode

```
START
    READ first number
    READ second number, which must be non-zero
```

```
    CALCULATE sum
    CALCULATE difference
    CALCULATE product
    CALCULATE quotient
    DISPLAY the results
STOP
```

Python implementation

Perform four arithmetic operations on two numbers.

```
print("BASIC ARITHMETIC CALCULATOR")
print("==========================")

first_number = float(input("Enter the first number: "))
second_number = float(input("Enter the second number (not
zero): "))

sum_result = first_number + second_number
difference = first_number - second_number
product = first_number * second_number
quotient = first_number / second_number

print()
print(f"Sum: {sum_result:.2f}")
print(f"Difference: {difference:.2f}")
print(f"Product: {product:.2f}")
print(f"Quotient: {quotient:.2f}")
```

Test cases

First number	Second number	Sum	Difference	Product	Quotient
10	5	15.00	5.00	50.00	2.00
7.5	2.5	10.00	5.00	18.75	3.00

-6	3	-3.00	-9.00	-18.00	-2.00

Discussion: Entering zero as the second number causes a `ZeroDivisionError`. In Week 3, we will use a conditional statement to prevent that calculation.

12. Guided project 2: Temperature converter

Filename: `temperature_converter.py`

Problem

Convert a temperature from Celsius to Fahrenheit.

Formula:

$$F = \left(C \times \frac{9}{5}\right) + 32$$

Input–process–output analysis

Input	Process	Output
Temperature in Celsius	Multiply by $9/5$, then add 32	Temperature in Fahrenheit

Pseudocode

START
  READ temperature in Celsius
  CALCULATE Fahrenheit AS (Celsius × 9 / 5) + 32
  DISPLAY temperature in Fahrenheit
STOP

Python implementation

Convert Celsius to Fahrenheit.celsius = float(input("Enter the temperature in Celsius: "))fahrenheit = (celsius * 9 / 5) + 32print(f"{celsius:.2f} degrees Celsius = " f"{fahrenheit:.2f} degrees Fahrenheit")

The final instruction spans two lines inside parentheses. Python combines the two adjacent f-strings into one message.

Test cases

Celsius	Expected Fahrenheit
0	32.00
25	77.00
100	212.00
-40	-40.00

Extension exercise

Create `fahrenheit_to_celsius.py` using:

$$C = (F - 32) \times \frac{5}{9}$$

Test with `32`, `77`, and `212` degrees Fahrenheit.

13. Guided project 3: Simple-interest calculator

Filename: `simple_interest.py`

Problem

Calculate simple interest and the final amount from a principal, annual interest rate, and duration in years.

Formulas:

$$I = \frac{P \times R \times T}{100}$$

$$A = P + I$$

Where:

- P is the principal.
- R is the annual percentage rate.
- T is the duration in years.
- I is the simple interest.
- A is the final amount.

If the rate is 10%, enter `10`, not `0.10`, because the formula already divides by 100.

Input–process–output analysis

Input	Process	Output
Principal, annual percentage rate, time in years	Calculate interest, then add it to principal	Interest and final amount

Pseudocode

```
START
   READ principal
   READ annual percentage rate
   READ time in years
   CALCULATE interest AS principal × rate × time / 100
   CALCULATE final amount AS principal + interest
   DISPLAY interest
   DISPLAY final amount
STOP
```

Python implementation

```
# Calculate simple interest using an annual percentage rate.print("SIMPLE-INTEREST CALCULATOR")print("==========================")principal = float(input("Enter the principal in naira: "))annual_rate = float(input("Enter the annual rate (%): "))time_years = float(input("Enter the time in years: "))interest = (principal * annual_rate * time_years) / 100final_amount = principal + interestprint()print(f"Principal: NGN {principal:,.2f}")print(f"Annual rate: {annual_rate:.2f}%")print(f"Time: {time_years:.2f} years")print(f"Simple interest: NGN {interest:,.2f}")print(f"Final amount: NGN {final_amount:,.2f}")
```

Worked example

For a principal of ₦50,000, an annual rate of 10%, and a duration of 2 years:

$$I = \frac{50{,}000 \times 10 \times 2}{100} = 10{,}000$$

$$A = 50{,}000 + 10{,}000 = 60{,}000$$

Expected calculated output:

Simple interest: NGN 10,000.00
Final amount: NGN 60,000.00

Extension exercise: Time supplied in months

Create a second version that accepts months:

time_months = float(input("Enter the time in months: "))time_years = time_months / 12

Then use `time_years` in the existing formula.

Check: With principal ₦50,000, rate 10%, and duration 6 months, the interest should be ₦2,500.

14. Common mistakes and corrections

Mistake	What happens	Correction
Adding two values returned directly by `input()`	The text is joined	Convert to `int` or `float`
Using `int()` for input such as `"3.5"`	`ValueError`	Use `float()` when decimals are allowed
Writing `student name`	Invalid variable name	Use `student_name`
Using a name before assigning it	`NameError`	Assign the value first

Writing `Length` after defining `length`	A different name is used	Match capitalisation
Using `^` for powers	Performs a different operation	Use `**`
Omitting parentheses in an average	Produces the wrong calculation	Use `(a + b + c) / 3`
Dividing by zero	`ZeroDivisionError`	Use a non-zero divisor this week
Forgetting `f` before an f-string	Braces are printed literally	Write `f"..."`
Entering `50,000` into `float(input(...))`	`ValueError`	Enter `50000` without commas

When asked for numeric input, enter the number alone. For example, enter `50000`, not `₦50,000` or `50000 naira`.

15. Independent practical exercises

Complete the following without copying an entire guided solution.

Exercise A: Classroom seating calculator

Ask the user for:

- Number of rows.
- Number of chairs per row.

Calculate and display the total number of chairs.

Use integers.

Test: Six rows with eight chairs per row should give `48`.

Exercise B: Three-score average calculator

Ask for three scores. Display:

- Total score.
- Average score to two decimal places.

Test: Scores `60`, `70`, and `80` should give total `210.00` and average `70.00`.

Exercise C: Shopping calculator

Ask for:

- Item name.
- Quantity.
- Unit price.

Calculate the total cost and display a purchase summary.

Test: Four pens at ₦250 each should cost ₦1,000.

Exercise D: Distance calculator

Ask for speed in kilometres per hour and travel time in hours.

Use:

$$Distance = Speed \times Time$$

Display the distance in kilometres.

Test: A speed of 60 km/h for 2.5 hours should give 150 km.

16. Assessed practical task

Title: Student Weekly Expense Calculator
Filename: `weekly_expenses.py`

Create a program that asks for:

- Student name.
- Daily transport cost.
- Daily feeding cost.

- Number of days on campus.
- Other expenses for the week.

Calculate:

$$Transporttotal = Dailytransportcost \times Days$$

$$Feedingtotal = Dailyfeedingcost \times Days$$

$$Weeklytotal = Transporttotal + Feedingtotal + Otherweeklyexpenses$$

Display a clear summary containing the student's name and all calculated totals. Format monetary values to two decimal places.

For this version, assume valid, non-negative input.

Sample test

Input	Value
Student name	Ada Okeke
Daily transport cost	1,000
Daily feeding cost	1,500
Days on campus	5
Other weekly expenses	2,000

Expected summary:

WEEKLY EXPENSE SUMMARY
Student: Ada Okeke
Transport total: NGN 5,000.00
Feeding total: NGN 7,500.00
Other expenses: NGN 2,000.00
Weekly total: NGN 14,500.00

Submission requirements

Submit:

1. The working Python file.
2. Input–process–output analysis.
3. Pseudocode.
4. Expected and actual results for two different tests.
5. A short explanation of why each input uses a string, integer, or float.

Suggested marking guide

Criterion	Marks
Correct inputs and appropriate type conversions	4
Correct calculations	5
Meaningful variable names	2
Clear output using f-strings	3
Input–process–output analysis and pseudocode	3
Two documented tests	2
Useful introductory comment	1
Total	**20**

17. Suggested laboratory delivery plan

Activity	Duration
Week 1 recap and folder preparation	10 minutes
Variables and data types	25 minutes

User input and type conversion	25 minutes
Arithmetic operators and precedence	25 minutes
Output formatting with f-strings	15 minutes
Guided calculator projects	40 minutes
Independent assessment	30 minutes
Review and submission check	10 minutes
Total	**180 minutes**

18. Review questions

1. What does the assignment operator `=` do?
2. Why is `student_name` valid but `student name` invalid?
3. What is the difference between `25` and `"25"`?
4. What data type does `input()` return?
5. When should you use `int()` rather than `float()`?
6. What is the output of `"12" + "3"`?
7. What is the difference between `/` and `//`?
8. What does `%` calculate?
9. Which operator calculates powers?
10. Why does `(60 + 70 + 80) / 3` need parentheses?
11. What does `:.2f` mean in an f-string?
12. Why must the time unit match the interest-rate period?

Preparation for Week 3: Keep all programs from this practical. Students will add conditional statements to check values, classify results, and prevent calculations such as division by zero.